Middle East Technical University | Department of Computer Engineering

CENG 477

Introduction to Computer Graphics

Fall '2026-2027
Assignment 1 - Ray Tracing
(v.1.0)

Due date: November 5, 2026, Thursday, 23:59

1 Objectives

Ray tracing is one of the core techniques of computer graphics. It has traditionally been used for rendering of offline content such as animation movies and realistic simulations. With the advent of modern GPUs, however, it is expected to soon become the primary method of rendering even for real-time content such as computer games. In this assignment, you are going to implement a basic ray tracer that simulates the propagation of light in real world to get an hands-on experience on this exciting rendering technique.

Keywords: *ray tracing, light propagation, geometric optics, ray-object intersections, shading*

2 Specifications

1. You should name your executable as "raytracer".
2. Your executable will take a JSON scene file as argument (e.g. "scene.json"). You should be able to run your executable via command "./raytracer scene.json".
3. You will save the resulting images in the ASCII PPM format, known as P3. The name of this image file will be defined in the scene file. This is a very simple and human-readable image format. The file starts with a 3-line header: the magic number `P3`, the image width and height, and the maximum color value of `255`. After the header, the R G B values of each pixel are written as integers separated by whitespace, row by row, starting from the top-left pixel. For example, a 2×1 image with one red and one green pixel is:

```
P3
2 1
255
255 0 0 0 255 0
```

The details of the PPM format can be found at: `http://netpbm.sourceforge.net/doc/ppm.html`

4. We will share several sample scenes with you that includes the input JSON files and their expected output images (PPM). You can use them to test your ray tracer. For grading, we can use extra scenes.

5. The scene file may contain multiple camera configurations. You should render as many images as the number of cameras. The output filenames for each camera is also specified in the scene file.

6. You will have at most 30 minutes to render scenes for each input file on inek machines. Programs exceeding this limit will be killed and will be assumed to have produced no image. Note that *horse_and_mug* scene is exempt from the 30-minute rule as it has a high object count, and it is optional to render.

7. You should use Blinn-Phong shading model for the specular shading computations.

8. You will implement two types of light sources: point and ambient. There may be multiple point light sources and a single ambient light. The values of these lights will be given as (R, G, B) color triplets that are not restricted to [0, 255] range (however, they cannot be negative as negative light does not make sense). Any pixel color value that is calculated by shading computations and is greater than 255 must be clamped to 255 and rounded to the nearest integer before writing it to the output PPM file.

9. Point lights will be defined by their intensity (power per unit solid angle). The irradiance due to such a light source falls off as inversely proportional to the squared distance from the light source. To simulate this effect, you must compute the irradiance at a distance of d from a point light as:

$$E(d) = \frac{I}{d^2},$$

where I is the original light intensity (a triplet of RGB values given in the JSON file) and $E(d)$ is the irradiance at distance d from the light source.

10. **Back-face culling** is a method used to accelerate the ray – object intersections by not computing intersections with triangles whose normals are pointing away from the camera. Its implementation is simple and done by calculating the dot product of the ray direction with the normal vector of the triangle. If the sign of the result is positive, then that triangle is ignored. Note that shadow rays should not use back-face culling. In this homework, back-face culling implementation is optional.

3 Scene File

The scene file will be formatted as a JSON file (see Section 11). In this file, there may be different numbers of materials, vertices, triangles, spheres, planes, cylinders, lights, and cameras. Cameras, lights, materials, and vertices are defined by a unique integer ID. The IDs for each type of element will start from one and increase sequentially. Also notice that, in the JSON file:

· Every number represented by X, Y and Z is a floating point number.

· Every number represented by R, G, B, and N is an integer.

· All values are given as **strings**. A string may hold several numbers separated by whitespace (e.g. `"Position": "0 5 5"`).

· Keys starting with an underscore are attributes of the element: `_id` is the ID of the element, `_type` is its type (e.g. `"mirror"`), and `_data` holds the actual data of `VertexData` and `Faces`.

· The `_id` of an object (`Mesh`, `Triangle`, `Sphere`, `Plane`, `Cylinder`) is only a label. It may be missing or may not be a number, and it is never referenced.

· If there is more than one element of the same kind (e.g. `Camera`, `PointLight`, `Material`, `Mesh`, `Sphere`), its value is an **array** of objects. If there is only one, its value **may be a single object** instead of an array. Your parser must handle both cases.

· Optional keys (e.g. `MirrorReflectance`, or any object type that is not used in the scene) may be missing from the file.

· Some sample files may contain extra keys that are not explained below (e.g. `NumSamples` in a camera). Your parser should simply ignore such keys.

Explanations for each JSON key are provided below:

- **BackgroundColor:** Specifies the R, G, B values of the background. If a ray sent through a pixel does not hit any object, the pixel will be set to this color. Only applicable for primary rays sent through pixels. If a reflection ray does not hit any object, it contributes no color, i.e. (0, 0, 0).
- **ShadowRayEpsilon:** When a ray hits an object, you are going to send a shadow ray from the intersection point to each point light source to decide whether the hit point is in shadow or not. Due to floating point precision errors, sometimes the shadow ray hits the same object even if it should not. Therefore, you must use this small ShadowRayEpsilon value, which is a floating point number, to move the intersection point a bit further from the hit point in the direction of the hit point's normal vector so that the shadow ray does not intersect with the same object again. Note that ShadowRayEpsilon value can also be used to avoid self-intersections while casting reflection rays from the intersection point.
- **MaxRecursionDepth:** Specifies how many bounces the ray makes off of mirror-like objects. Applicable only when a material is of type mirror. Primary rays are assumed to start with zero bounce count.
- **Camera:**

  ·**Position** parameters define the coordinates of the camera.
  ·**Gaze** parameters define the direction that the camera is looking at. You must assume that the Gaze vector of the camera is always perpendicular to the image plane.
  ·**Up** parameters define the up vector of the camera.

·**NearPlane** defines the coordinates of the image plane with Left, Right, Bottom, Top floating point parameters, respectively.
·**NearDistance** defines the distance of the image plane to the camera.
·**ImageResolution** defines the resolution of the image with Width and Height integer parameters, respectively.
·**ImageName** defines the name of the output file.

Cameras defined in this homework will be right-handed. The mapping of Up and Gaze vectors to the camera terminology used in the course slides is given as:

$$\begin{aligned} \text{Up} &= v \\ \text{Gaze} &= -w \\ u &= v \times w \end{aligned}$$

- **AmbientLight:** is defined by just an X, Y, Z radiance triplet. This is the amount of light received by each object even when the object is in shadow. Color channel order of this triplet is RGB.

- **PointLight:** is defined by a position and an intensity, which are all floating point numbers. Color channel order of intensity is RGB.

- **Material:** A material can be defined with ambient, diffuse, specular, and mirror reflectance properties for each color channel. The values are floats between 0.0 and 1.0, and color channel order is RGB.
  PhongExponent defines the specularity exponent in Blinn-Phong shading.
  MirrorReflectance represents the degree of the mirrorness of the material. If the material is of type mirror, you must cast new rays and scale the resulting color value with the MirrorReflectance parameters. The attribute `"_type": "mirror"` is provided only for the materials which has non-zero MirrorReflectance.

- **VertexData:** The `_data` string contains all vertices one after another. Every three numbers form a vertex whose x, y, and z coordinates are given as floating point values, respectively. The first vertex's ID is 1.

- **Mesh:** Each mesh is composed of several faces. A face is actually a triangle which contains three vertices. When defining a mesh, the `_data` string of the Faces key contains all faces one after another. Every three integers form a triangle, given as vertex IDs in counter-clockwise order (see Triangle explanation below). Material key represents the material ID of the mesh.

- **Triangle:** A triangle is represented by Material and Indices keys. Material represents the material ID. Indices are the integer vertex IDs of the vertices that construct the triangle. Vertices are given in counter-clockwise order, which is important when you want to calculate the normals of the triangles. Counter-clockwise order means that if you close your right-hand following the order of the vertices, your thumb points in the direction of the surface normal.

- **Sphere:** A sphere is represented by Material, Center, and Radius keys. Material represents the material ID. Center represents the vertex ID of the point which is the center of the sphere. Radius is the radius of the sphere.

- **Plane:** A plane is represented by Material, Center, and Normal keys. Material is the material ID. Center is the integer vertex ID of a point lying on the plane. Normal is a 3D vector giving the plane's outward-facing normal (it need not be unit; you should normalize it). The plane has an infinite extent.
- **Cylinder:** A finite right circular cylinder is represented by Material, Center, Axis, Radius, and Height keys. Material is the material ID. Center is the integer vertex ID of the cylinder's geometric center (midpoint along its axis). Axis is a 3D vector pointing from the center toward the top cap (it need not be unit; you should normalize it). Radius is the cylinder radius, and Height is the distance between the two caps (the cylinder extends $\frac{Height}{2}$ in both directions along the axis from Center). The cylinder is closed, i.e. it has a top cap and a bottom cap.

Ray–cylinder intersection concept: We treat the cylinder using its natural frame: a center C, an axis direction $\mathbf{A}$ (normalized), a radius r, and a height h (caps at $\pm h/2$ along $\mathbf{A}$). Given a ray $O + t\mathbf{D}$, we split motion into two pieces: the part *along* the axis and the part *perpendicular* to it.

- **Side test (infinite tube).** Project the ray onto the plane perpendicular to $\mathbf{A}$. In that 2D view, the cylinder becomes a circle of radius r. Intersect the projected ray with this circle (a simple quadratic). This yields up to two candidate distances t where the ray would hit the *infinite* tube. For each candidate, check the hit point's axial coordinate (distance along $\mathbf{A}$) and keep it only if it lies within the finite height interval $[-h/2, h/2]$. Those that pass are "lateral" hits.
- **Cap tests (two disks).** Intersect the ray with the two planes orthogonal to $\mathbf{A}$ at the top and bottom of the cylinder ($\pm h/2$ along $\mathbf{A}$). For each plane hit, check whether the hit point is inside the circular cap (distance to the cap center $\leq r$).
- **Choose the earliest valid hit.** Among all valid candidates (side and caps), pick the smallest positive t (in front of the camera) and return it together with the part tag, so that you can compute the correct normal (radial for the side, $\pm\mathbf{A}$ for the caps).
- For additional background, you can benefit from NYU Media Lab and Wikipedia.

4 JSON Parsing

In this homework, you are expected to parse the JSON file yourself. You are allowed to use AI for this purpose. If you want to parse it manually, a very popular library is *nlohmann* C++ JSON parser: `https://github.com/nlohmann/json`.

5 PPM Output

Similar to JSON parsing, in this homework, you are expected to write code to output a PPM image in P3 format yourself. The format is very simple, so it is a straightforward task. You are also allowed to use AI for this purpose.

6 AI Policy

The AI policy for this homework is that you can use it generate code for you only for the JSON parsing and PPM output stages. Ray tracing code should be your own. Using AI to generate code for ray equation, intersection, shading, vector operations, etc. that are all critical graphics tasks is not allowed. Our lecture slides comprehensively cover all the related topics and potential pitfalls to serve as a reference for you during coding. Your codes will be checked manually and using automated tools to ensure that you comply with this policy.

7 Group Policy

This homework can be done in groups of two student. This is not because the homework is too difficult for a single person – it is simply there to let you openly discuss with each other and write code together to have a better learning experience. Each student in the group must have full understanding of the submitted code. Each student is expected to know all concepts and take responsibility for the entirety of the submitted code. To ensure that this is the case, students may be called for random interviews between the submission and grade release.

8 Hints & Tips

1. Start early. It takes time to get a ray tracer up and running. Note that this year you also need to write the scene parser and the PPM writer yourself.
2. You may use the -O3 option while compiling your code for optimization. This itself will provide a huge performance improvement.
3. Try to pre-compute anything that would be used multiple times and save these results. For example, you can pre-compute the normals of the triangles and save it in your triangle data structure when you read the input file.
4. If you see generally correct but noisy results (black dots), it is most likely that there is a floating point precision error (you may be checking for exact equality of two FP numbers instead of checking if they are within a small epsilon).
5. For debugging purposes, consider using low resolution images. Also it may be necessary to debug your code by tracing what happens for a single pixel (always simplify the problem when debugging).
6. We will not test your code with strange configurations such as the camera being inside a sphere or a cylinder, a point light being inside a sphere or a cylinder, or with objects in front of the image plane. If you doubt whether a special case in addition to these will be tested, you may ask this on ODTUClass.

9 Bonus

1. You can get up to 10 bonus if you create and share interesting JSON scene files. These should not be very simple scenes as we already share such scenes with you. Interesting scenes to get bonus will be determined by the course instructors and assistants.

10 Regulations

1. **Programming Language:** C/C++. You also must use gcc/g++ for the compiler. Any C++ version can be used as long as the code works on Inek machines. Only under special cases other PLs may be allowed (for instance being an Erasmus student and not having learned these languages in your program). Ask your instructor if in doubt.

2. **Provided Files:** No code template is provided. We only provide sample scenes: their input JSON files and the expected output PPM images. All of the code (including the scene parser and the PPM writer) must be written by you.

3. **Additional Libraries:** If you are planning to use any library other than *(i)* the standard library of the language and *(ii)* pthread, please first ask about it on ODTUClass and get a confirmation, as any unapproved library usage may cause a penalty during grading.

4. **Submission:** Submission will be done via ODTUClass. To submit, you must select an option in "HW Groups" page in ODTUClass according to the criteria in the Groups section below. Create a "**tar.gz**" file named "raytracer.tar.gz" that contains all your source code files and a Makefile. The executable should be named as "raytracer" and should be able to be run using the following commands (scene.json will be provided by us during grading):

   tar -xf raytracer.tar.gz
   make
   ./raytracer scene.json

   Any error in these steps will cause a 10 points penalty over 100 points!

5. **Groups:** You can team-up with another student. To form a group, both students must select the same group with 2 people capacity in "HW Groups" page on ODTUClass. If you want to work on the homework by yourself, in the same page select any "Individual Work" option which only have 1 person capacity. A selection must be made to upload the homework. See the group policy defined above.

6. **Previous Submissions:** Our goal is to ensure equal contribution between partners. If you previously took this course and scored above 50 on an assignment, you may only work with a partner this semester if you complete the assignment entirely from scratch. You cannot reuse or adapt any code from previous semesters. If you intend to reference or reuse past submissions, you and your partner must each submit individual work instead.

7. **Late Submission:** You can submit your codes up to 3 days late. Each late day will be deduced from the total 7 credits for the semester. However, if you fail to submit even after 3 days, you will get 0 regardless of how many late credits you may have left. If you submit late and still get zero, you cannot claim back your late days. You must e-mail the assistants if you want your submission not to be evaluated (and therefore preserve your late day credits).

8. **Cheating: We have zero tolerance policy for cheating**. People involved in cheating will be punished according to the university regulations and will get 0 from the homework. You can discuss algorithmic choices, but sharing code between groups or using third party code is strictly forbidden.

To prevent cheating in this homework, we also compare your codes with online ray tracers and previous years' student solutions. We also use various tools to detect AI usage. In case a match is found, this will also be considered as cheating. Even if you take only a "part" of the code from somewhere or somebody else, this is also cheating. Please be aware that there are very good tools that detect similarities and AI usage in codes – so please don't take such paths.

Use of AI tools: In this homework, you can use AI generated code only for JSON parsing and PPM output stages. Other than these, the code you submit must be written by you (and your partner). Using AI tools to write, complete, fix, or rewrite your code counts as using third party code and will be treated as cheating. If you use such tools at all, use them only to understand concepts, never to produce code. You must be able to explain every line of your submission; we may ask you to do so, and failing to explain your own code will also be considered as cheating.

9. **Forum:** Any updates/corrections and discussions regarding the homework will be on ODTU-Class. You should check it on a daily basis.

10. **Evaluation:** Your codes will be evaluated on Inek machines using various scene files, including but not limited to the examples provided. A subset of the given test cases will be used in grading (not necessarily all of them). We will not use automated grading; outputs will be inspected visually, so tiny pixel-value differences will not cause point loss. Rendering all evaluated scenes correctly within the time limit will earn you **100 points**.

11 Sample Scene File

```
{
  "Scene": {
    "BackgroundColor": "R G B",
    "ShadowRayEpsilon": "X",
    "MaxRecursionDepth": "N",
    "Cameras": {
      "Camera": [
        {
          "_id": "Cid",
          "Position": "X Y Z",
          "Gaze": "X Y Z",
          "Up": "X Y Z",
          "NearPlane": "Left Right Bottom Top",
          "NearDistance": "X",
          "ImageResolution": "Width Height",
          "ImageName": "ImageName.ppm"
        }
      ]
    },
    "Lights": {
      "AmbientLight": "X Y Z",
      "PointLight": [
        {
          "_id": "Lid",
          "Position": "X Y Z",
          "Intensity": "X Y Z"
        }
      ]
    },
    "Materials": {
      "Material": [
        {
          "_id": "Mid",
          ["_type": "mirror",]
          "AmbientReflectance": "X Y Z",
          "DiffuseReflectance": "X Y Z",
          "SpecularReflectance": "X Y Z",
          "MirrorReflectance": "X Y Z",
          "PhongExponent": "X"
        }
      ]
    },
    "VertexData": {
      "_data": "V_{1X} V_{1Y} V_{1Z} V_{2X} V_{2Y} V_{2Z} ...",
      "_type": "xyz"
    },
    "Objects": {
      "Mesh": [
        {
          "_id": "Meid",
          "Material": "N",
          "Faces": {
            "_data": "F_{1_V1} F_{1_V2} F_{1_V3} F_{2_V1} F_{2_V2} F_{2_V3} ...",
            "_type": "triangle"
          }
        }
      ],
      "Triangle": [
```

```
            {
              "_id": "Tid",
              "Material": "N",
              "Indices": "$V_1$ $V_2$ $V_3$"
            }
          ],
          "Sphere": [
            {
              "_id": "Sid",
              "Material": "N",
              "Center": "N",
              "Radius": "X"
            }
          ],
          "Plane": [
            {
              "Material": "N",
              "Center": "N",
              "Normal": "X Y Z"
            }
          ],
          "Cylinder": [
            {
              "_id": "Cyid",
              "Material": "N",
              "Center": "N",
              "Axis": "X Y Z",
              "Radius": "X",
              "Height": "X"
            }
          ]
        }
      }
    }
}
```